

Dynamic Priority Exchange Server for Aperiodic Task Scheduling

Md Ratul Ahmed Alvi
Real-Time Systems Seminar
Hochschule Hamm-Lippstadt

Abstract—Real-time systems often execute hard periodic tasks together with soft aperiodic requests. The periodic workload must meet its deadlines, while irregular requests should receive a short response time. The Dynamic Priority Exchange (DPE) server addresses this problem under Earliest Deadline First (EDF) scheduling by reserving execution capacity for aperiodic work and preserving unused capacity through deadline exchanges. This paper presents the task model, formal exchange rules, schedulability condition, resource-reclaiming mechanism, and implementation overhead of DPE. A compact UPPAAL model is used to demonstrate the reclaiming and later consumption of unused periodic execution time. The verified properties cover deadlock freedom, eventual request completion, and consistent capacity transfer. The model is intentionally limited to a main DPE feature rather than a complete EDF implementation.

Index Terms—real-time systems, dynamic priority exchange, EDF, aperiodic scheduling, resource reclaiming, UPPAAL

I. INTRODUCTION

Real-time applications commonly combine periodic control activities with requests that arrive at irregular times. Periodic tasks may represent sensing or control loops and normally have hard deadlines. Aperiodic requests may represent operator commands, diagnostics, or display updates. Their deadlines are soft, but a long response time can reduce the usefulness of the system. Executing such requests only in the background protects periodic tasks but often gives poor responsiveness.

Server-based scheduling reserves a controlled amount of processor time for aperiodic work. The Dynamic Priority Exchange server applies this idea under EDF and avoids discarding reserved capacity when no request is waiting [1]. It allows a periodic job with a later deadline to execute and stores an equal amount of aperiodic capacity at that deadline level. The capacity can later be reclaimed when an aperiodic request arrives.

This paper explains the mathematical model and formal DPE rules, illustrates the exchange mechanism, discusses schedulability and overhead, and presents a small UPPAAL implementation of resource reclaiming. An automotive control example is used to connect the algorithm with a practical application.

II. POSITION OF DPE AMONG APERIODIC SERVERS

The simplest way to execute soft aperiodic work is background service. It requires almost no additional scheduler mechanism because an aperiodic request executes only when no periodic job is ready. The disadvantage is that response

time becomes directly dependent on the unused processor intervals of the periodic schedule. A polling server improves this situation by activating periodically with a reserved budget. However, if no request is ready at the polling instant, the capacity can be lost even if a request arrives shortly afterward.

Priority Exchange was introduced to preserve server capacity in fixed-priority systems [3]. Rather than losing unused high-priority budget, a lower-priority periodic task executes and the server receives equivalent capacity at the lower priority. DPE applies this principle to EDF, where priority is represented by an absolute deadline rather than by a fixed task level. This is why the algorithm stores several capacity values associated with different deadlines.

The same source paper also introduces Dynamic Sporadic, Total Bandwidth, EDL, and Improved Priority Exchange servers [1]. These mechanisms answer the same basic question in different ways: how much processor time can be safely assigned to aperiodic work, and at what deadline should that work execute? Total Bandwidth assigns a deadline directly from the execution demand and a reserved bandwidth. EDL uses computed idle time and gives optimal response time but has high complexity. Improved Priority Exchange combines offline idle-time information with exchange-based reclaiming.

DPE is therefore not presented as the fastest mechanism in every situation. Its importance for this seminar lies in the clear relationship between EDF priority, server capacity, and deadline exchange. The method also provides a direct utilization test and a resource-reclaiming rule that can be represented with timed automata.

III. SYSTEM MODEL AND MATHEMATICAL BASIS

A periodic task τ_i is represented by its period T_i and worst-case execution time (WCET) C_i . The source model assumes a preemptive uniprocessor, relative deadlines equal to periods, synchronous first releases, and known WCET values. Aperiodic requests have unknown arrival times, but their execution times are known when they arrive [1].

The utilization of task τ_i and the total periodic utilization are

$$U_i = \frac{C_i}{T_i}, \quad U_P = \sum_{i=1}^n \frac{C_i}{T_i}. \quad (1)$$

The DPE server has period T_S , execution capacity C_S , and utilization

$$U_S = \frac{C_S}{T_S}. \quad (2)$$

TABLE I
CORE SCHEDULER STATE REQUIRED BY DPE.

Item	Purpose
Periodic job	Release, absolute deadline, remaining execution
Aperiodic queue	Pending soft requests and remaining execution
Capacity entry	Remaining budget and associated absolute deadline
Server release	Creates C_S at each server period
EDF selector	Chooses the earliest job or positive capacity

EDF is a dynamic-priority policy: among all ready jobs, the job with the earliest absolute deadline executes first. For independent preemptive periodic tasks on one processor, EDF can schedule total utilization up to one under the stated assumptions [2].

DPE introduces capacity entries in addition to normal jobs. Let C_S^d denote server capacity with deadline d , and let $C_{S_i}^{d_i}$ denote capacity stored at the absolute deadline d_i of a periodic job. Positive capacities participate in the same EDF ordering as jobs. Thus, an earlier capacity deadline means a higher service priority.

IV. DYNAMIC PRIORITY EXCHANGE

A. Exchange Rule

At the beginning of each server period, a fresh capacity C_S is created with the current server deadline. Whenever an aperiodic-capacity entry is the highest-priority entity, DPE applies one of three rules [1]:

- 1) If a request is pending, it executes until it completes or the active capacity is exhausted.
- 2) If no request is pending but periodic work is ready, the periodic job with the earliest deadline executes. For an execution interval Δ , the capacity values are updated as

$$C_S^d \leftarrow C_S^d - \Delta, \quad (3)$$

$$C_{S_i}^{d_i} \leftarrow C_{S_i}^{d_i} + \Delta. \quad (4)$$

This is the actual priority exchange: periodic work runs earlier, while the same processor time remains available to aperiodic work at deadline d_i .

- 3) If no request and no periodic job are ready, the processor is idle and the active capacity is consumed.

Ties between a job and a capacity with the same deadline are resolved in favor of the capacity. Capacity is therefore preserved only while it can still be used without delaying the periodic workload beyond the schedule assumed by EDF.

Table I summarizes the minimum state that a scheduler must maintain. The important difference from a conventional EDF ready queue is that a capacity entry has a deadline and remaining budget but no independent computation. Its budget is consumed either by an aperiodic request, by an exchange with periodic execution, or by idle time.

B. Worked Exchange Example

The example in [1] uses

$$\tau_1 = (T_1, C_1) = (8, 2), \quad \tau_2 = (12, 3), \quad (T_S, C_S) = (6, 3). \quad (5)$$

At $t = 0$, the server owns three units with deadline 6. Since no aperiodic request is waiting, τ_1 executes for two units and transfers two capacity units to deadline 8. Then τ_2 executes for one unit and transfers one unit to deadline 12. When the capacity at deadline 8 later becomes the highest-priority entity, the remaining two units of τ_2 execute and transfer those units to deadline 12. The same deadline-12 capacity entry therefore grows from one unit to three units. Additional server periods and an idle interval then elapse before the request arrives, while usable capacity is preserved at later deadline levels.

At $t = 14$, a seven-unit aperiodic request arrives. It is served by two units with deadline 18, two preserved units with deadline 24, and three new units replenished at $t = 18$. Fig. 1 redraws the important capacity movements rather than copying the original schedule.

C. Schedulability and Reclaiming

The central DPE schedulability result is

$$U_P + U_S \leq 1. \quad (6)$$

The proof transforms a DPE schedule into an ordinary EDF schedule. Periodic execution performed during an exchange is conceptually postponed to the interval in which the corresponding stored capacity is consumed. The server is then equivalent to an additional periodic task with parameters (T_S, C_S) . Because exchanged periodic work executes no later in the real DPE schedule than in the transformed schedule, feasibility of the transformed EDF schedule implies feasibility of the DPE schedule [1].

The necessity of (6) follows from the opposite operating condition. If enough aperiodic work is continuously available, the server consumes C_S units during every interval of length T_S . In that case, it is indistinguishable from an ordinary periodic task. Therefore, a total utilization greater than one cannot be feasible on one processor. The result is consequently both sufficient and necessary under the assumptions of Section II.

For the task set $C_1 = 1$, $T_1 = 5$, $C_2 = 2$, $T_2 = 10$, and a server with $C_S = 2$, $T_S = 5$,

$$U_P = \frac{1}{5} + \frac{2}{10} = 0.4, \quad U_S = \frac{2}{5} = 0.4, \quad U_P + U_S = 0.8. \quad (7)$$

The utilization test is satisfied. Increasing C_S to 4 would produce a total utilization of 1.2, so the theorem would no longer guarantee the periodic deadlines.

DPE can also reclaim unused periodic execution time. If a task with WCET C_i completes after actual time $c_i < C_i$, the spare capacity

$$C_{\text{spare}} = C_i - c_i \quad (8)$$

is added to the capacity entry at the same deadline. This is safe because the processor time was already included in the original WCET-based guarantee [1].

V. IMPLEMENTATION AND UPPAAL EVALUATION

A. Runtime Overhead

DPE requires the scheduler to manage both jobs and capacity entries. During an exchange, two capacity values are

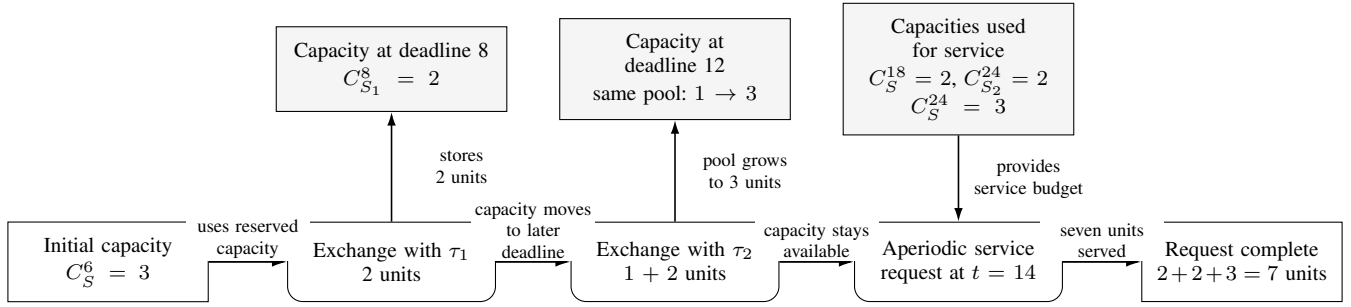


Fig. 1. Simplified DPE capacity flow based on the example in [1]. When no aperiodic request is pending, periodic tasks execute while the reserved capacity is preserved at later deadline levels. The deadline-12 entry is one capacity pool that grows from one to three units. At $t = 14$, the seven-unit request is served using capacities of two, two, and three units.

updated using (3) and (4), and the running capacity must be checked for exhaustion. The ready queue may contain one capacity entry for each periodic-job instance and can therefore become approximately twice as long. Spuri and Buttazzo argue that linear-list operations can at most roughly double, while the increase is practically negligible with a binary heap [1]. Thus, the algorithm is not trivial, but its run-time overhead remains comparable to EDF bookkeeping.

If the EDF ready structure contains m entities, selection and update cost depends on its implementation. A simple ordered or unordered linear list gives $O(m)$ insertion or selection in the worst case. A binary heap gives $O(\log m)$ insertion and removal of the earliest deadline. DPE does not change these asymptotic queue bounds; it increases m by introducing capacity entries and adds constant-time arithmetic for each exchange tick. This distinction is useful: the algorithm adds state and more frequent updates, but it does not require a fundamentally different priority-ordering operation.

B. Model Structure

UPPAAL supports the modeling and verification of networks of timed automata [4]. The developed model contains a periodic automaton P , an aperiodic automaton A , and a server automaton S . It intentionally represents the reclaiming path rather than the complete multi-deadline DPE scheduler.

The periodic automaton executes until clock p reaches the configured actual execution time. It then calculates

$$\text{stored_capacity} = \text{WCET} - \text{ACTUAL} \quad (9)$$

and assigns the periodic deadline to the stored capacity. Automaton A changes from waiting to ready at $AP_ARRIVAL$. Automaton S starts service only when the periodic task has finished, a request is pending, and stored capacity is positive. At that point, the value is transferred to active_capacity and the two automata synchronize through the service channels. The automata are presented separately to keep their labels readable. Fig. 2 shows the periodic task, Fig. 3 shows the aperiodic request, and Fig. 4 shows the server.

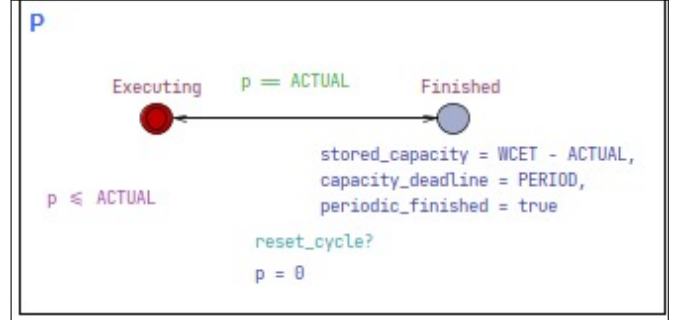


Fig. 2. Periodic-task automaton P . Early completion stores the difference between WCET and actual execution time.

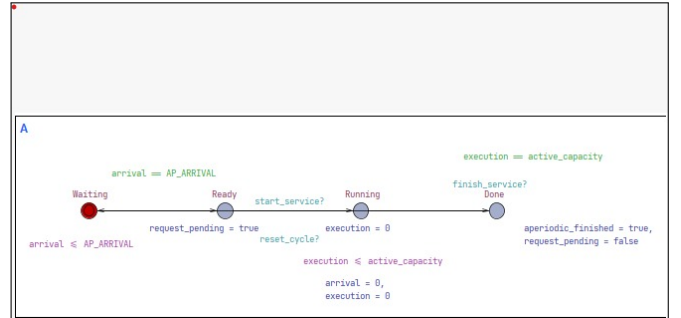


Fig. 3. Aperiodic-request automaton A , from arrival and readiness to service and completion.

C. Verified Properties and Scope

The verifier successfully evaluated the following properties:

- $A \Box \text{not deadlock,}$
- $A \lt \> A.\text{Done,}$
- $A \Box S.\text{Serving} \Rightarrow \text{active_capacity} > 0,$
- $A \Box S.\text{Serving} \Rightarrow \text{stored_capacity} = 0,$
- $A \Box A.\text{Done} \Rightarrow \text{active_capacity} = 0,$
- $A \Box \text{periodic_finished} \Rightarrow \text{capacity_deadline} = \text{PERIOD.}$

The first property establishes deadlock freedom and the second checks eventual completion of the request. The remaining invariants ensure that service uses positive capacity, stored capacity is transferred before service, no active capacity remains

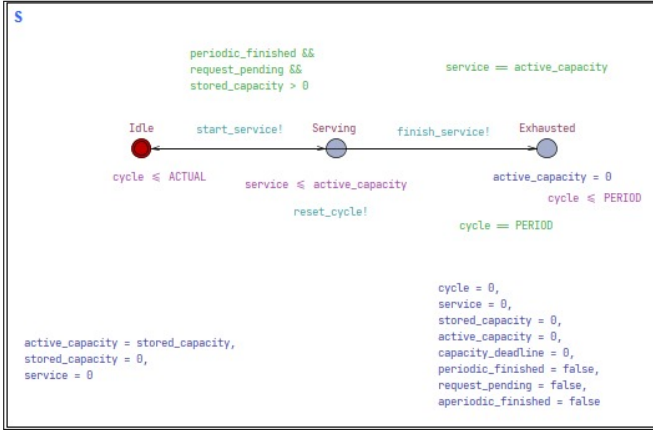


Fig. 4. Server automaton S , which transfers stored capacity and synchronizes with the aperiodic request.

TABLE II
INTERPRETATION OF THE VERIFIED UPPAAL PROPERTIES.

Property group	Meaning
Deadlock freedom	At least one legal transition or delay remains possible
Reachability of $A.Done$	The modeled request can complete
Positive active budget	Service never starts without reclaimed capacity
Stored budget cleared	Capacity is transferred exactly once
Zero budget after done	The modeled service consumes the active budget
Deadline preservation	Reclaimed time keeps the periodic deadline level

after completion, and the reclaimed budget keeps the periodic deadline.

The quantifiers have different purposes. The prefix $A[]$ expresses a universal invariant: the stated condition must hold in every reachable state along every execution path. The query $A <> A.Done$ is a universal liveness requirement: every possible model execution must eventually reach the completed state. Using both kinds of query prevents a misleading verification in which capacity rules are safe on reachable states but the request can remain waiting forever. Table II summarizes the role of each property group, while Fig. 5 shows the verifier confirming all six queries.

The model contains one periodic task, one aperiodic request, and one stored deadline. It therefore verifies resource reclaiming and capacity transfer, but not EDF ordering among several periodic jobs or repeated exchanges among several deadline levels. A complete DPE model would need a ready queue, recurring server replenishment, several capacity entries, and deadline-based dispatching. This limitation is stated explicitly so that the verification results are not interpreted as proof of the entire DPE algorithm.

D. Implementation Assessment

The model was kept small for two reasons. First, the resource-reclaiming path can be observed directly: early completion creates capacity, arrival enables the server, and synchronization completes the request. Second, a small state

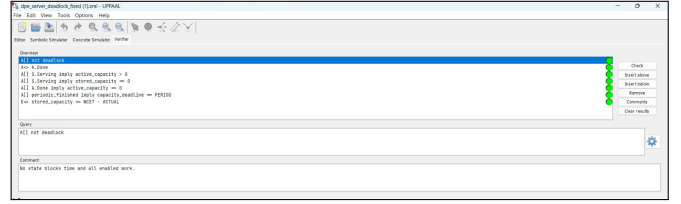


Fig. 5. UPPAAL verifier confirming the six selected safety and liveness properties.

TABLE III
ILLUSTRATIVE AUTOMOTIVE PERIODIC WORKLOAD.

Activity	T_i (ms)	C_i (ms)
Wheel-speed processing	10	1
Brake-pressure control	20	4
Stability monitoring	50	5

space makes the verified properties easy to relate to individual transitions. The model is therefore useful as an executable explanation of Section III-C.

The abstraction also reveals what is still required for a full implementation. Several periodic automata would need independent releases and absolute deadlines. The server would need recurring replenishments, and a scheduler automaton would have to compare every ready job with every positive capacity entry. Because capacity can migrate through several deadline levels, the number of entries can grow with the number of active periodic instances. These additions are technically possible in UPPAAL, but they would verify a different and substantially larger model.

VI. APPLICATION AND DISCUSSION

An automotive electronic control unit provides a suitable example. Table III defines a simple periodic workload.

The periodic utilization is

$$U_P = \frac{1}{10} + \frac{4}{20} + \frac{5}{50} = 0.4. \quad (10)$$

A server with $T_S = 10$ ms and $C_S = 2$ ms has $U_S = 0.2$, producing a total utilization of 0.6. The task set therefore satisfies (6) and leaves additional processor margin.

A diagnostic request may arrive when a wheel-speed sensor reports an unusual value. If no diagnostic request is pending when server capacity becomes available, DPE can let a lower-priority control job execute while preserving the same capacity at its deadline. The later diagnostic request can then use the preserved budget instead of waiting for background time.

Resource reclaiming provides an additional benefit. If the braking task has a WCET of 4 ms but completes in 2 ms, then (8) yields 2 ms of reclaimed capacity. Since that time had already been reserved for the braking task, using it at the same deadline does not add processor demand.

DPE provides better aperiodic responsiveness than background and polling service, but the source paper reports slightly weaker performance than later mechanisms such as Total Bandwidth and Improved Priority Exchange servers [1]. Table IV summarizes this qualitative trade-off.

TABLE IV
QUALITATIVE COMPARISON REPORTED IN [1].

Mechanism	Response	Implementation
Background/Polling	Low	Very simple
DPE	Improved	Moderate bookkeeping
Total Bandwidth	Very good	Very low overhead
Improved Priority	Near optimal	More memory/state
Exchange		

Its main strength is the clear EDF utilization guarantee combined with reusable capacity. Its main cost is extra scheduler state and capacity bookkeeping. The usefulness of preserved capacity can also decrease after it is transferred to a later deadline because the corresponding priority becomes lower.

A. Validity and Extension Path

The mathematical result and the UPPAAL experiment answer different questions. Equation (6) is a theorem for the complete DPE algorithm under the stated task assumptions. The model-checking results instead establish properties only for the implemented network of automata. They show that the particular reclaiming design is internally consistent, but they do not replace the schedulability proof. Keeping these two evidence types separate avoids claiming more from the model than it verifies.

The application parameters are also illustrative rather than measurements from a production braking controller. Their purpose is to demonstrate how periodic and server utilization are calculated. A real deployment would require measured or statically bounded WCET values, an analysis of release jitter, interrupt handling, and shared-resource blocking. If tasks access common resources, the utilization-only test must be extended with an appropriate resource-access protocol and blocking analysis.

A next UPPAAL version should add a second periodic task and represent absolute deadlines explicitly. A scheduler automaton could maintain two job entries and multiple capacity

entries, selecting the minimum deadline at each dispatch point. Repeated server releases would then test whether capacity is replenished correctly and whether an arriving request can consume capacity stored at different deadline levels. The most important additional properties would state that every periodic job completes before its deadline and that the sum of stored, active, consumed, and exchanged capacity is conserved. Such a model would move from demonstrating resource reclaiming toward validating the complete DPE exchange mechanism.

VII. CONCLUSION

The Dynamic Priority Exchange server serves soft aperiodic requests while protecting hard periodic tasks scheduled by EDF. Its defining operation is a one-to-one exchange between early periodic execution and aperiodic capacity stored at the periodic deadline. The resulting system is schedulable under the condition $U_P + U_S \leq 1$, and unused periodic WCET can be reclaimed safely.

The UPPAAL model demonstrates a limited but central implementation feature: spare periodic time is stored, activated only for a pending request, and consumed consistently. Verification confirmed deadlock freedom and eventual completion within this abstraction. DPE therefore offers a practical balance between responsiveness and schedulability, although a complete implementation requires more bookkeeping than ordinary EDF and a richer model than the presented resource-reclaiming prototype.

REFERENCES

- [1] M. Spuri and G. C. Buttazzo, "Scheduling aperiodic tasks in dynamic priority systems," *Real-Time Systems*, vol. 10, no. 2, pp. 179–210, Mar. 1996.
- [2] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.
- [3] J. P. Lehoczky, L. Sha, and J. K. Strosnider, "Enhanced aperiodic responsiveness in hard real-time environments," in *Proc. IEEE Real-Time Systems Symp.*, 1987, pp. 261–270.
- [4] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a nutshell," *Int. J. Softw. Tools Technol. Transfer*, vol. 1, nos. 1–2, pp. 134–152, 1997.